

Открыть в при

ться

Войти

Mediu

Вы вышли из аккаунта. Войдите в систему, используя свою учетную запись (**sv__@g__.com**), чтобы просматривать другие истории, доступные только участникам. Войти



Skill Stuff · [Подписаться на публикацию](#)

Чистый код +

Принцип чистого кода +

Программирование +

Разработка программного обеспечения +

10 правил чистого кода, которые я никогда не нарушаю

5 мин на чтение · 19 июля 2026



Шаян Хан

Подписаться

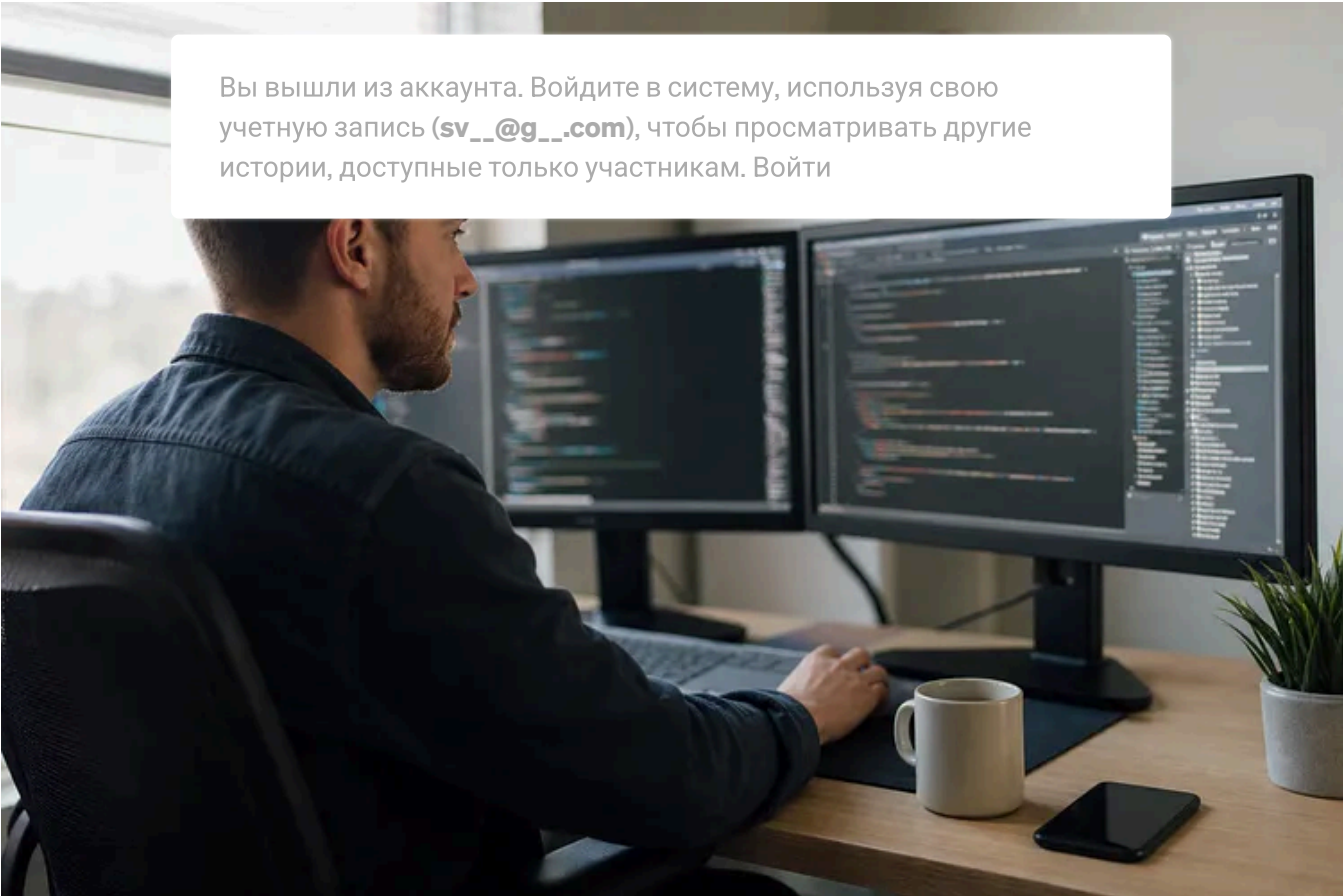


Слушать



Поделиться

Простые привычки, которые облегчают чтение, исправление и публикацию кода даже спустя месяцы



Вы вышли из аккаунта. Войдите в систему, используя свою учетную запись (**sv__@g__.com**), чтобы просматривать другие истории, доступные только участникам. Войти

ЧИСТЫЙ КОД

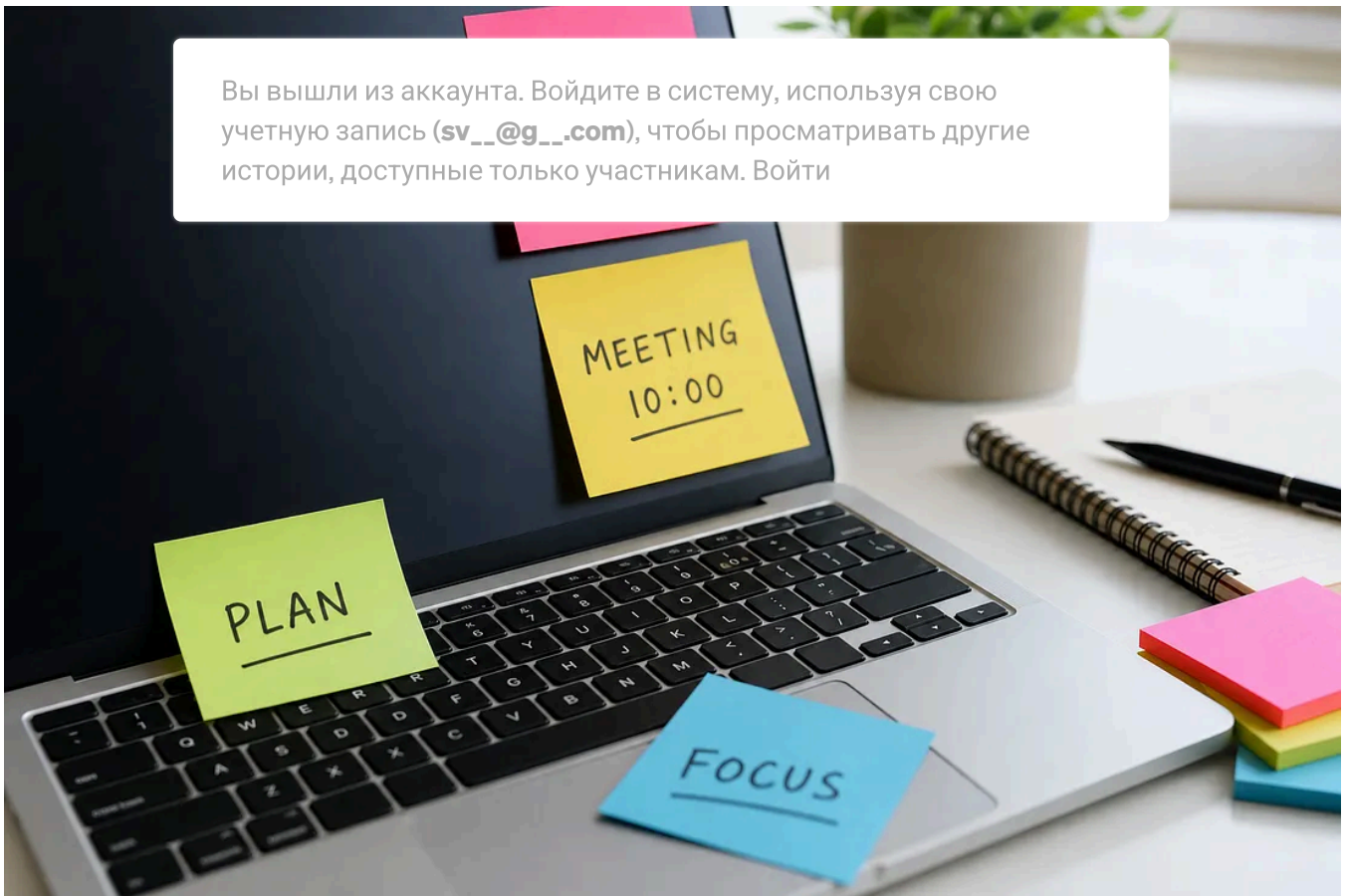
Я работал над кодом, которым гордился. А еще я работал над кодом, из-за которого мне хотелось закрыть ноутбук и пойти прогуляться.

Разница почти никогда не заключалась в каких-то хитростях. Она заключалась в нескольких скучных правилах, которых я придерживался, когда уставал, торопился или «просто хотел поскорее что-то выпустить». В такие дни в код прокрадывается беспорядок.

Вот 10 правил чистого кода, которые я не нарушаю. Не потому, что так написано в книге. Потому что каждый раз, когда я их нарушал, я расплачивался за это позже — обычно в 11 вечера, когда обнаруживал ошибку, которую не мог найти.

1. Называйте вещи так, чтобы их мог прочитать человек.

Вы вышли из аккаунта. Войдите в систему, используя свою учетную запись (sv__@g__.com), чтобы просматривать другие истории, доступные только участникам. Войти



Если переменная называется `d`, `temp`, или `data2`, то в будущем вы будете ненавидеть себя из прошлого.

Я называю вещи так, чтобы их мог прочитать следующий человек, который откроет файл. Через три недели этим человеком часто оказываюсь я, совершенно не помня, что я имел в виду.

Плохо:

```
const d = getU();
```

Лучше:

```
const activeUsers = getActiveUsers();
```

Хорошее название заменяет комментарий. Плохое название порождает тайну.

Вы вышли из аккаунта. Войдите в систему, используя свою учетную запись (**sv__@g__.com**), чтобы просматривать другие истории, доступные только участникам. Войти

2. Делай!



Если функция выполняет пять задач, она может дать сбой в пяти аспектах — и вы заметите только два из них.

Мое правило: одна функция — одна четкая задача. Если в описании появляется «а также», я разбиваю функцию на части.

Когда функция разрастается настолько, что я уже не могу держать ее в голове, я перестаю ее дополнять. Я разбиваю ее на части. Маленькие части легче тестировать, переименовывать и удалять, когда идея меняется.

3. Не оставляйте комментарии в духе «я разберусь с этим позже»

Мы все пишем `// TODO: fix this`. Я тоже так делаю.

Чего я не делаю, так это оставляю эти заметки на несколько месяцев. TODO, которое т: Вы вышли из аккаунта. Войдите в систему, используя свою . Код
выглядит учетную запись (sv__@g__.com), чтобы просматривать другие истории, доступные только участникам. Войти

Если я не могу исправить это сейчас, я либо:

- создаю настоящий тикет, либо
- сегодня исправляю самую безопасную часть

Hope is not a cleanup plan.

4. Write code for readers, not for showing off

Clever code feels good for about ten minutes. Then someone has to change it.

I avoid tricks that need a TED talk to explain. Clear beats clever. Every time.

If I need a comment to prove how smart the code is, the code is probably doing too much. I'd rather write something plain that a junior can follow in one pass.

5. Repeat yourself a little before you abstract too early

Copy-paste gets a bad reputation. Blind abstraction is worse.

I've built "helpful" shared helpers that ended up serving three different use cases poorly. Then every change broke something unrelated.

Get Shayan Khan's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

My rule now: wait until I see the same idea clearly three times. Then extract. Not after one c

Вы вышли из аккаунта. Войдите в систему, используя свою учетную запись (**sv__@g__.com**), чтобы просматривать другие истории, доступные только участникам. Войти

6. Make failure obvious

Silent failures are the worst kind of bugs. The app “works,” until it doesn’t, and nobody knows why.

I return clear errors. I log what failed. I don’t swallow exceptions and move on like nothing happened.

If something goes wrong, the next developer should be able to see it without digging through five files and guessing.

7. Keep formatting consistent

Tabs vs spaces. Quote style. Import order. Line length.

None of this makes you a genius. All of it makes a codebase feel calm.

I let a formatter do the boring work. Same rules for everyone. No debates in pull requests about commas. We save that energy for logic that matters.

8. Delete dead code without guilt

Old functions that “we might need someday” almost never get used again. They just sit there and confuse people.

If it’s unused, I delete it. Git remembers. I can bring it back if I truly need it.

Dead code is not a backup. It’s clutter with a story nobody trusts anymore.

9. Leave t

I don't rew
behind.

Вы вышли из аккаунта. Войдите в систему, используя свою
учетную запись (**sv__@g__.com**), чтобы просматривать другие
истории, доступные только участникам. Войти

ovement

Rename a confusing variable. Split a chunky function. Remove a leftover comment.
Tiny fixes add up.

Big rewrites are rare. Small care, every day, is what keeps a project from rotting.

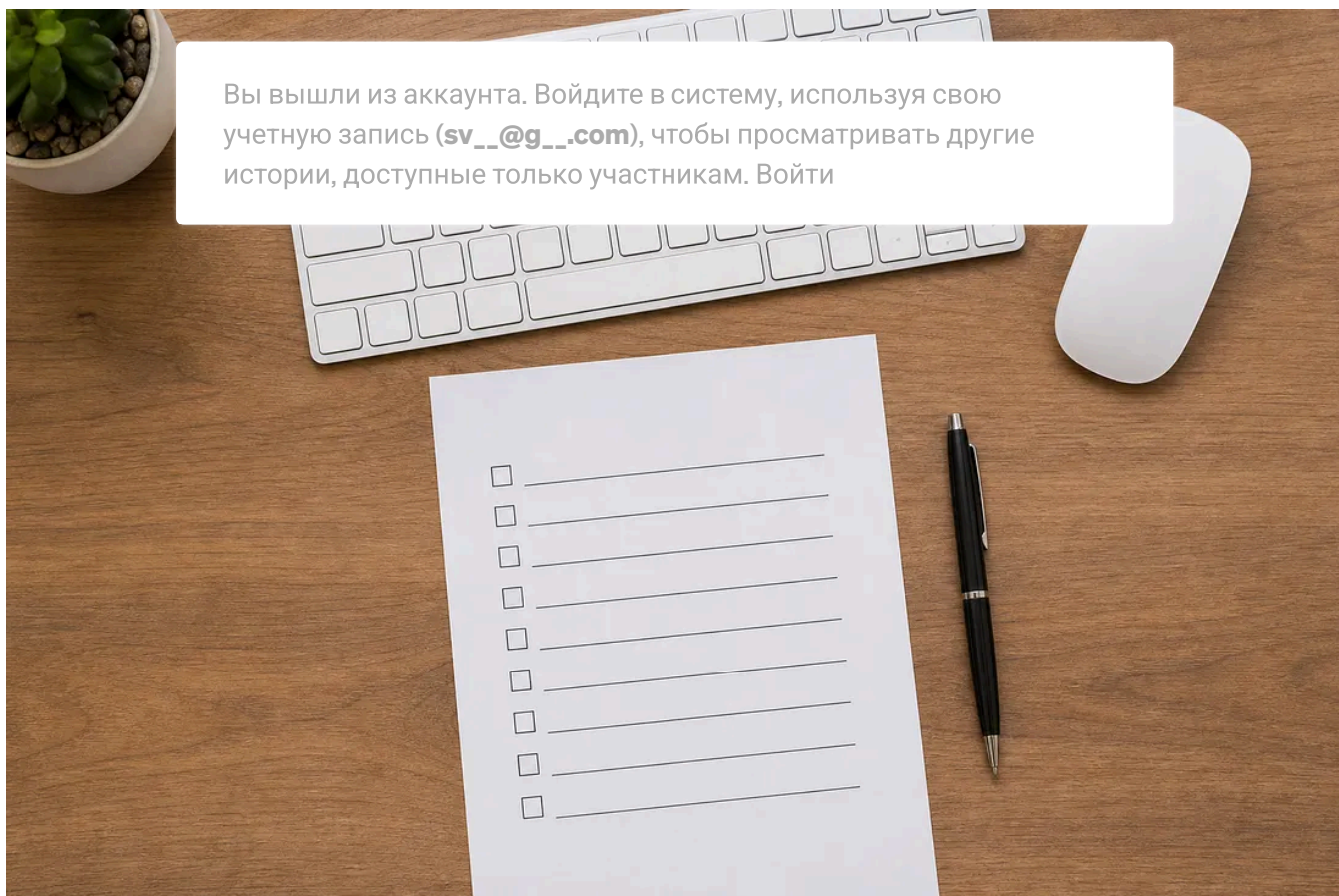
10. Tests for the parts that scare you

I don't aim for 100% coverage theater. I write tests for the spots that break business
logic or cost money when they fail.

Payments. Auth. The weird edge case that already burned us once.

A few honest tests beat a mountain of empty ones. If a change makes me nervous,
that's my signal to lock it down.

A quick checklist I keep nearby



Вы вышли из аккаунта. Войдите в систему, используя свою учетную запись (**sv__@g__.com**), чтобы просматривать другие истории, доступные только участникам. Войти

Before I open a pull request, I ask myself:

1. Can I explain this change in one sentence?
2. Would a teammate understand the names without asking me?
3. Did I leave any “temporary” mess?
4. Is there one scary path I should cover with a test?
5. Did I make the file a little better than I found it?

If I can say yes to most of those, I’m good to ship.

Final thought

Clean code isn’t about being perfect. It’s about being kind to the next person including the version of you who has to fix this on a Friday.

Pick two rules from this list and stick to them for a week. Don’t try to become a new developer overnight. Small habits stick. Big speeches don’t.

If this helped, share it with a teammate who keeps saying “we’ll clean it up later.”

Later has a Вы вышли из аккаунта. Войдите в систему, используя свою учетную запись (sv__@g__.com), чтобы просматривать другие истории, доступные только участникам. Войти

- Clean Code
- Clean Code Principle
- Programming
- Software Development



Follow

Published in Skill Stuff

749 followers · Last published 2 hours ago

Skill Stuff is your go-to hub for coding tips, problem-solving strategies, and developer growth insights.



Follow

Written by Shayan Khan

31 followers · 20 following

UI/UX Designer & Web App Developer focused on designing clean, scalable enterprise dashboards and ERP systems

Вы вышли из аккаунта. Войдите в систему, используя свою учетную запись (**sv__@g__.com**), чтобы просматривать другие истории, доступные только участникам. Войти

Вы вышли из аккаунта. Войдите в систему, используя свою учетную запись (**sv__@g__.com**), чтобы просматривать другие истории, доступные только участникам. Войти